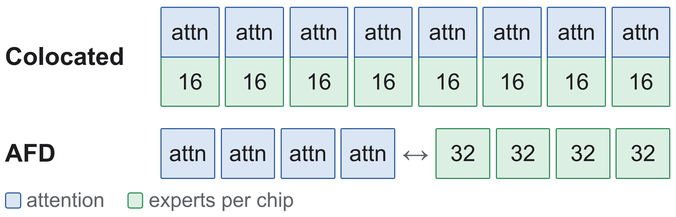
Attention–FFN disaggregation on TPU v6e

Charlie Xue, Akhil Chilaka, Rohan Anne, Jerry Ji

Motivation

- In Qwen3-30B-A3B, attention reads each user's KV cache, so its cost grows with the number of users, while the 128 experts are 58 GB of weights that every user shares.
- Colocated serving runs both on every chip, so they can't be scaled or batched separately.
- AFD puts them on different chips, as MegaScale-Infer and Step-3 already do on GPUs.



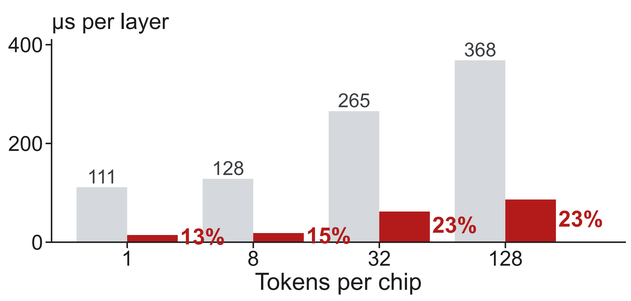
The obstacle on TPU

torch_tpu compiles one program and runs it on every chip, so attention chips and expert chips can't run different code. We tried five ways around this:

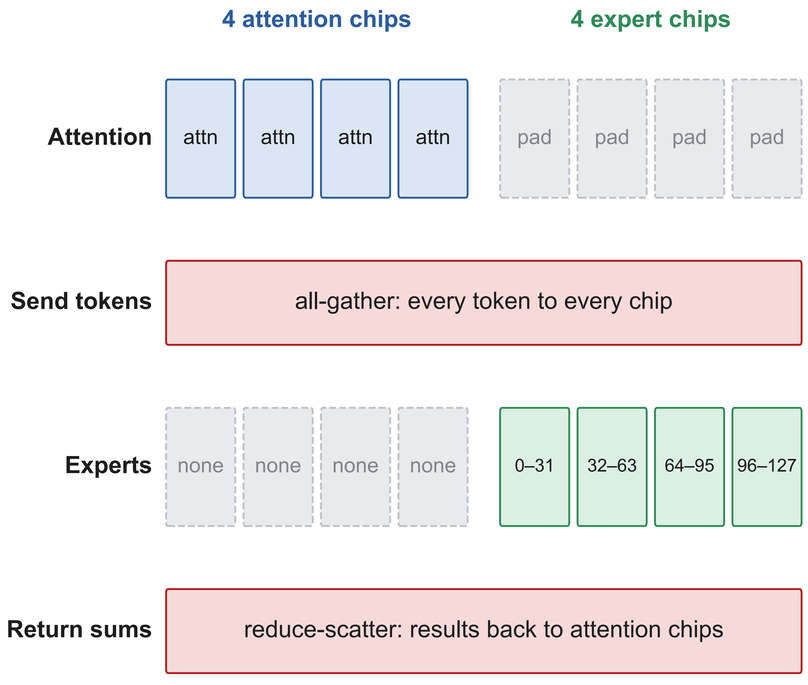
✗	A separate program per group	hangs
✗	A runtime branch (`torch.cond`)	unsupported
✗	Uneven all-to-all	won't compile
✗	Chip-to-chip send and receive	1.3 ms per layer
✓	**One program, role set by data**	**15 to 90 µs**

Moving tokens is cheap

We timed the working approach on its own and found that sending tokens to the expert chips and back (**red**) takes 13 to 23% of the time of a whole colocated layer (grey).



Our design



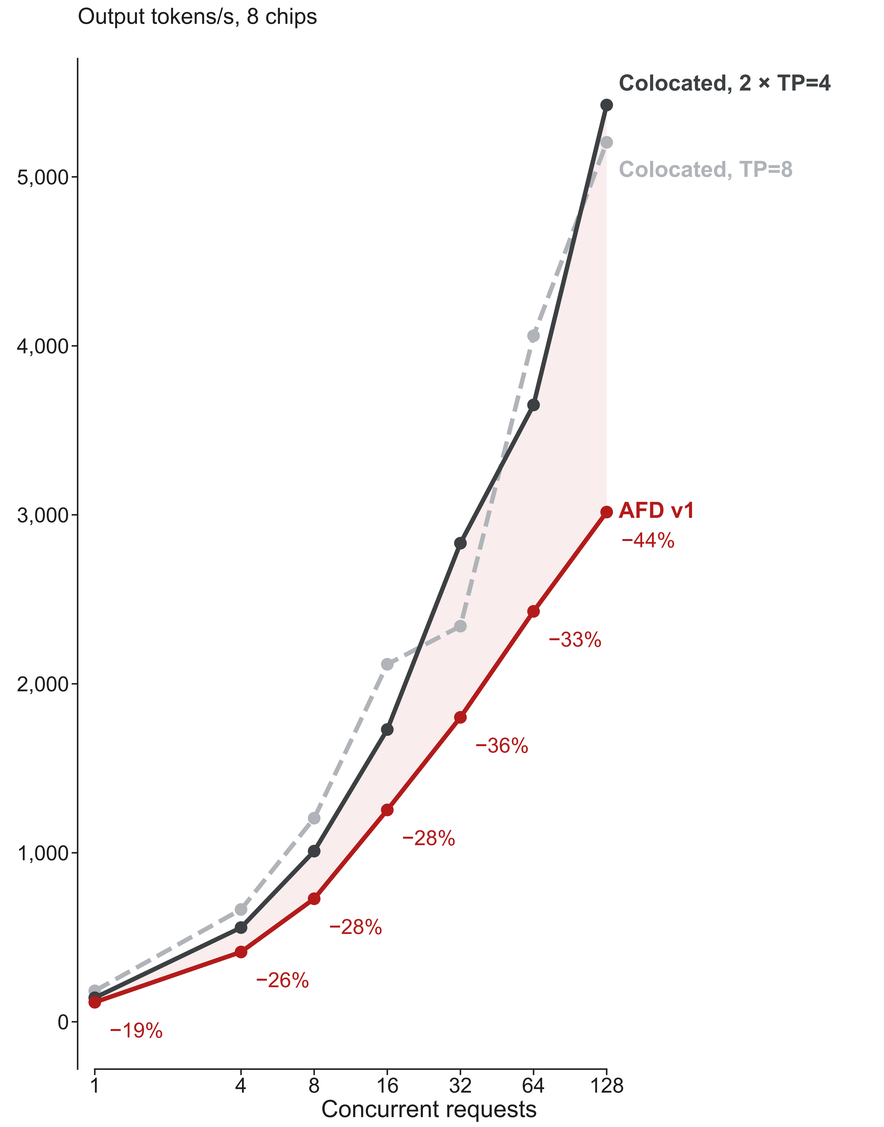
- With data parallelism, vLLM runs two copies of the model on 4 chips each, and both copies execute the same compiled program in lockstep.
- We send every request to the first copy and give its chips an empty expert range, which turns them into attention chips that skip the expert work.
- The second copy gets no requests, so its chips act as expert chips, each holding 32 of the 128 experts.
- In every layer, tokens travel to the expert chips and back through vLLM's existing all-gather and reduce-scatter, so we didn't need new communication code.
- The grey cells show wasted work, since the expert chips run attention on padding batches and each group waits while the other works.

AFD gives correct output

AFD gets all 4 of our known-answer prompts right, and where its text differs from colocated serving, the two candidate tokens had similar probabilities:

prompt	AFD's token	colocated's token
Apollo 11	**marked** .239	**achieved** .239
fibonacci(n)	**1** .525	**0** .463

AFD works, but its throughput is 19 to 44% below 2 × TP=4



Since moving tokens is cheap, most of this gap probably comes from the idle time in the grey cells, which we plan to confirm by profiling a single layer.

Next steps

- Profile one layer to see where the time goes
- Remove the padding work on the expert chips
- Split each batch in two so both groups work at once
- Compare accuracy on GSM8K and try other chip splits